Here's the complete prompt ready to copy-paste into your AI-powered IDE (Cursor, GitHub Copilot, Replit AI, etc.):

---

text

DJANGO TIMETABLE MANAGEMENT SYSTEM - COMPLETE PROJECT SPECIFICATION

Create a complete Django web application for intelligent university timetable management (Gestion Automatique des Emplois du Temps Universitaires). Use Object-Oriented Programming principles, Django Template Language (DTL) forms, and real-time notifications. All user-facing content must be in French, technical code in English.

PROJECT REQUIREMENTS

Technology Stack
- Django 5.0+
- Python 3.10+ (strict OOP principles)
- SQLite for development
- Django Channels 4.0+ with Redis for real-time notifications
- Django Template Language (DTL) with Bootstrap 5
- WebSockets for live updates

User Roles (5 types)
1. **Administrateur** - Full system management
2. **Enseignant** - Teacher with reservation rights
3. **Étudiant** - Student schedule viewing
4. **Association** - Club/association with reservation rights (like teacher)
5. **Invité** - Guest/public access without login

Public Access Features
- Landing page accessible without authentication
- Browse general schedules and room availability
- Registration forms for Students and Associations
- Contact form to reach administrators
- No login required for public browsing

DATABASE MODELS (OOP Design)

1. User Model (extends Django User)
```python
class User(AbstractUser):
    role = models.CharField(max_length=20, choices=[
        ('admin', 'Administrateur'),
        ('teacher', 'Enseignant'),
        ('student', 'Étudiant'),
        ('association', 'Association'),
        ('guest', 'Invité')
```

```
    ])
    phone = models.CharField(max_length=20, blank=True)

    class Meta:
        verbose_name = "Utilisateur"
        verbose_name_plural = "Utilisateurs"
```

2. Teacher (Enseignant)

python

```
class Teacher(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    specialization = models.CharField(max_length=100, verbose_name="Spécialité")
    department = models.CharField(max_length=100, verbose_name="Département")
    employee_id = models.CharField(max_length=20, unique=True)

    def get_weekly_hours(self):
        """Calculate total weekly teaching hours"""
        pass

    class Meta:
        verbose_name = "Enseignant"
        verbose_name_plural = "Enseignants"
```

3. Student (Étudiant)

python

```
class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    student_id = models.CharField(max_length=20, unique=True)
    enrollment_year = models.IntegerField(verbose_name="Année d'inscription")
    group = models.ForeignKey('Group', on_delete=models.SET_NULL, null=True)

    class Meta:
        verbose_name = "Étudiant"
        verbose_name_plural = "Étudiants"
```

4. Association (NEW)

python

```
class Association(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    name = models.CharField(max_length=200, verbose_name="Nom de l'association")
    description = models.TextField(verbose_name="Description")
    president_name = models.CharField(max_length=100, verbose_name="Président")
    email = models.EmailField(verbose_name="Email")
    phone = models.CharField(max_length=20, verbose_name="Téléphone")
    is_approved = models.BooleanField(default=False, verbose_name="Approuvée")
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    class Meta:
        verbose_name = "Association"
        verbose_name_plural = "Associations"
```

5. Program (Filière)

python

```
class Program(models.Model):
    name = models.CharField(max_length=200, verbose_name="Nom")
    code = models.CharField(max_length=20, unique=True)
    department = models.CharField(max_length=100, verbose_name="Département")
    degree_level = models.CharField(max_length=50, choices=[
        ('licence', 'Licence'),
        ('master', 'Master'),
        ('doctorat', 'Doctorat')
    ])

    class Meta:
        verbose_name = "Filière"
        verbose_name_plural = "Filières"
```

6. Group (Groupe)

python

```
class Group(models.Model):
    name = models.CharField(max_length=100, verbose_name="Nom du groupe")
    program = models.ForeignKey(Program, on_delete=models.CASCADE)
    academic_year = models.CharField(max_length=10, verbose_name="Année universitaire")
    capacity = models.IntegerField(verbose_name="Capacité")

    class Meta:
        verbose_name = "Groupe"
        verbose_name_plural = "Groupes"
```

7. Room (Salle)

python

```
class Room(models.Model):
    name = models.CharField(max_length=50, verbose_name="Nom")
    room_type = models.CharField(max_length=20, choices=[
        ('classe', 'Salle de classe'),
        ('amphitheatre', 'Amphithéâtre'),
        ('labo', 'Laboratoire'),
        ('salle_info', 'Salle informatique')
    ], verbose_name="Type")
    capacity = models.IntegerField(verbose_name="Capacité")
    building = models.CharField(max_length=50, verbose_name="Bâtiment")
    floor = models.IntegerField(verbose_name="Étage")
    equipment = models.ManyToManyField('Equipment', blank=True)
    is_active = models.BooleanField(default=True)
```

```
    def check_availability(self, start_time, end_time):
        """Check if room is available in time range"""
        conflicts = Session.objects.filter(
            room=self,
            start_datetime__lt=end_time,
            end_datetime__gt=start_time
        )
        return not conflicts.exists()

    class Meta:
        verbose_name = "Salle"
        verbose_name_plural = "Salles"
```

8. Equipment (Équipement)

python

```
class Equipment(models.Model):
    name = models.CharField(max_length=100, verbose_name="Nom")
    description = models.TextField(blank=True, verbose_name="Description")

    class Meta:
        verbose_name = "Équipement"
        verbose_name_plural = "Équipements"
```

9. Session (Séance)

python

```
class Session(models.Model):
    session_type = models.CharField(max_length=20, choices=[
        ('cours', 'Cours'),
        ('td', 'TD'),
        ('tp', 'TP'),
        ('examen', 'Examen')
    ], verbose_name="Type")
    subject = models.CharField(max_length=200, verbose_name="Matière")
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)
    room = models.ForeignKey(Room, on_delete=models.SET_NULL, null=True)
    groups = models.ManyToManyField(Group, verbose_name="Groupes")
    start_datetime = models.DateTimeField(verbose_name="Début")
    end_datetime = models.DateTimeField(verbose_name="Fin")
    is_exam = models.BooleanField(default=False, verbose_name="Est un examen")
    is_validated = models.BooleanField(default=False, verbose_name="Validée")

    def save(self, *args, **kwargs):
        is_new = self.pk is None
        super().save(*args, **kwargs)
```

```
        # Trigger notifications if exam
        if is_new and self.is_exam:
            from apps.notifications.services import NotificationService
            NotificationService.notify_students_exam_scheduled(self)

    class Meta:
        verbose_name = "Séance"
        verbose_name_plural = "Séances"
```

10. RoomReservationRequest (Demande de Réservation)

python

```
class RoomReservationRequest(models.Model):
    requester_type = models.CharField(max_length=20, choices=[
        ('teacher', 'Enseignant'),
        ('association', 'Association')
    ])
    teacher = models.ForeignKey(Teacher, null=True, blank=True, on_delete=models.CASCADE)
    association = models.ForeignKey(Association, null=True, blank=True, on_delete=models.CASCADE)
    room = models.ForeignKey(Room, on_delete=models.CASCADE)
    requested_datetime = models.DateTimeField(verbose_name="Date et heure")
    duration = models.IntegerField(verbose_name="Durée (heures)")
    reason = models.TextField(verbose_name="Motif")
    status = models.CharField(max_length=20, choices=[
        ('pending', 'En attente'),
        ('approved', 'Approuvée'),
        ('rejected', 'Rejetée')
    ], default='pending', verbose_name="Statut")
    is_exam = models.BooleanField(default=False, verbose_name="Pour un examen")
    created_at = models.DateTimeField(auto_now_add=True)

    def approve(self, admin_user):
        """Approve reservation and send notifications"""
        self.status = 'approved'
        self.save()

        from apps.notifications.services import NotificationService

        # Notify requester
        requester = self.teacher.user if self.teacher else self.association.user
        NotificationService.send_notification(
            user=requester,
            notification_type='reservation_approved',
            title='Réservation approuvée',
            message=f'Votre réservation de la salle {self.room.name} a été approuvée.',
            related_object=self
```

```
        )

        # Notify students if teacher reservation
        if self.teacher:
            NotificationService.notify_students_reservation_approved(self)

    class Meta:
        verbose_name = "Demande de Réservation"
        verbose_name_plural = "Demandes de Réservation"
```

11. TeacherUnavailability (Indisponibilité)

python

```
class TeacherUnavailability(models.Model):
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)
    start_datetime = models.DateTimeField(verbose_name="Début")
    end_datetime = models.DateTimeField(verbose_name="Fin")
    reason = models.TextField(verbose_name="Raison")

    class Meta:
        verbose_name = "Indisponibilité Enseignant"
        verbose_name_plural = "Indisponibilités Enseignants"
```

12. Notification (NEW - Real-time)

python

```
class Notification(models.Model):
    recipient = models.ForeignKey(User, on_delete=models.CASCADE, related_name='notifications')
    notification_type = models.CharField(max_length=50, choices=[
        ('reservation_approved', 'Réservation Approuvée'),
        ('reservation_rejected', 'Réservation Rejetée'),
        ('exam_scheduled', 'Examen Programmé'),
        ('timetable_updated', 'Emploi du Temps Modifié'),
        ('room_changed', 'Salle Modifiée'),
        ('session_cancelled', 'Séance Annulée'),
        ('contact_received', 'Message Reçu')
    ])
    title = models.CharField(max_length=200, verbose_name="Titre")
    message = models.TextField(verbose_name="Message")
    related_object_type = models.CharField(max_length=50, null=True, blank=True)
    related_object_id = models.IntegerField(null=True, blank=True)
    is_read = models.BooleanField(default=False, verbose_name="Lu")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = "Notification"
        verbose_name_plural = "Notifications"
        ordering = ['-created_at']
```

13. ContactMessage (NEW)

python

```python
class ContactMessage(models.Model):
    sender_name = models.CharField(max_length=100, verbose_name="Nom")
    sender_email = models.EmailField(verbose_name="Email")
    user = models.ForeignKey(User, null=True, blank=True, on_delete=models.SET_NULL)
    subject = models.CharField(max_length=200, verbose_name="Sujet")
    message = models.TextField(verbose_name="Message")
    status = models.CharField(max_length=20, choices=[
        ('pending', 'En attente'),
        ('in_progress', 'En cours'),
        ('resolved', 'Résolu')
    ], default='pending', verbose_name="Statut")
    response = models.TextField(blank=True, verbose_name="Réponse")
    created_at = models.DateTimeField(auto_now_add=True)
    responded_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        verbose_name = "Message de Contact"
        verbose_name_plural = "Messages de Contact"
```

14. Timetable (Emploi du Temps)

python

```python
class Timetable(models.Model):
    program = models.ForeignKey(Program, on_delete=models.CASCADE)
    academic_year = models.CharField(max_length=10, verbose_name="Année universitaire")
    semester = models.CharField(max_length=20, choices=[
        ('S1', 'Semestre 1'),
        ('S2', 'Semestre 2')
    ])
    sessions = models.ManyToManyField(Session)
    is_published = models.BooleanField(default=False, verbose_name="Publié")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Emploi du Temps"
        verbose_name_plural = "Emplois du Temps"
```

CORE FEATURES IMPLEMENTATION

Feature 1: BULK ROOM CREATION (Admin)

Form:

python

```python
class BulkRoomCreateForm(forms.Form):
    building = forms.CharField(max_length=50, label="Bâtiment")
    room_prefix = forms.CharField(max_length=10, label="Préfixe (ex: B)", help_text="Lettre ou code du bâtiment")
```

```
    start_number = forms.IntegerField(label="Numéro de début", initial=1, min_value=1)
    end_number = forms.IntegerField(label="Numéro de fin", min_value=1)
    room_type = forms.ChoiceField(choices=Room.ROOM_TYPE_CHOICES, label="Type de salle")
    capacity = forms.IntegerField(label="Capacité", min_value=1)
    floor = forms.IntegerField(label="Étage")
    equipment = forms.ModelMultipleChoiceField(
        queryset=Equipment.objects.all(),
        widget=forms.CheckboxSelectMultiple,
        required=False,
        label="Équipements disponibles"
    )

    def clean(self):
        cleaned_data = super().clean()
        start = cleaned_data.get('start_number')
        end = cleaned_data.get('end_number')

        if start and end and start > end:
            raise forms.ValidationError("Le numéro de début doit être inférieur au numéro de fin.")

        return cleaned_data
```

View:

python

```
class BulkRoomCreateView(LoginRequiredMixin, UserPassesTestMixin, FormView):
    form_class = BulkRoomCreateForm
    template_name = 'core/bulk_room_create.html'
    success_url = reverse_lazy('room_list')

    def test_func(self):
        return self.request.user.role == 'admin'

    def form_valid(self, form):
        prefix = form.cleaned_data['room_prefix']
        start = form.cleaned_data['start_number']
        end = form.cleaned_data['end_number']
        building = form.cleaned_data['building']
        room_type = form.cleaned_data['room_type']
        capacity = form.cleaned_data['capacity']
        floor = form.cleaned_data['floor']
        equipment_list = form.cleaned_data['equipment']

        # Create rooms in bulk
        rooms_to_create = []
        for num in range(start, end + 1):
```

```
        rooms_to_create.append(Room(
            name=f"{prefix}{num}",
            building=building,
            room_type=room_type,
            capacity=capacity,
            floor=floor,
            is_active=True
        ))

    # Use Django's bulk_create for efficiency
    created_rooms = Room.objects.bulk_create(rooms_to_create)

    # Add equipment to all created rooms
    for room in created_rooms:
        room.equipment.set(equipment_list)

    messages.success(
        self.request,
        f"{len(created_rooms)} salles créées avec succès ({prefix}{start} à {prefix}{end})"
    )
    return super().form_valid(form)
```

DTL Template (bulk_room_create.html):

xml

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}

{% block content %}
<div class="container mt-4">
  <div class="row">
    <div class="col-md-8 offset-md-2">
      <div class="card">
        <div class="card-header bg-primary text-white">
          <h3><i class="fas fa-door-open"></i> Création en Masse de Salles</h3>
        </div>
        <div class="card-body">
          <p class="text-muted">
            Créez plusieurs salles identiques en une seule opération.
            Par exemple : B1 à B23 créera 23 salles (B1, B2, B3... B23).
          </p>

          <form method="post" novalidate>
            {% csrf_token %}

            <div class="row">
```

```
    <div class="col-md-6">
        {{ form.building|as_crispy_field }}
    </div>
    <div class="col-md-6">
        {{ form.room_prefix|as_crispy_field }}
    </div>
</div>

<div class="row">
    <div class="col-md-6">
        {{ form.start_number|as_crispy_field }}
    </div>
    <div class="col-md-6">
        {{ form.end_number|as_crispy_field }}
    </div>
</div>

<div class="alert alert-info">
    <i class="fas fa-info-circle"></i>
    <span id="preview-text">Aucune salle sélectionnée</span>
</div>

<div class="row">
    <div class="col-md-4">
        {{ form.room_type|as_crispy_field }}
    </div>
    <div class="col-md-4">
        {{ form.capacity|as_crispy_field }}
    </div>
    <div class="col-md-4">
        {{ form.floor|as_crispy_field }}
    </div>
</div>

<div class="mb-3">
    <label class="form-label">Équipements disponibles</label>
    {{ form.equipment }}
</div>

<div class="d-grid gap-2">
    <button type="submit" class="btn btn-primary btn-lg">
        <i class="fas fa-plus-circle"></i> Créer les Salles
    </button>
    <a href="{% url 'room_list' %}" class="btn btn-secondary">
```

```
                    <i class="fas fa-times"></i> Annuler
                  </a>
                </div>
              </form>
            </div>
          </div>
        </div>
      </div>
</div>

<script>
document.addEventListener('DOMContentLoaded', function() {
    const prefix = document.getElementById('id_room_prefix');
    const start = document.getElementById('id_start_number');
    const end = document.getElementById('id_end_number');
    const preview = document.getElementById('preview-text');

    function updatePreview() {
        const p = prefix.value || 'X';
        const s = parseInt(start.value) || 1;
        const e = parseInt(end.value) || 1;
        const count = Math.max(0, e - s + 1);

        if (count > 0) {
            preview.textContent = `Création de ${count} salles : ${p}${s} à ${p}${e}`;
        } else {
            preview.textContent = 'Numéros invalides';
        }
    }

    prefix.addEventListener('input', updatePreview);
    start.addEventListener('input', updatePreview);
    end.addEventListener('input', updatePreview);
    updatePreview();
});
</script>
{% endblock %}
```

Feature 2: REAL-TIME NOTIFICATION SYSTEM

Install dependencies in requirements.txt:

```
text
channels>=4.0
channels-redis>=4.1
daphne>=4.0
redis>=5.0
```

ASGI Configuration (asgi.py):

python

```python
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from channels.security.websocket import AllowedHostsOriginValidator

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

django_asgi_app = get_asgi_application()

from apps.notifications import routing

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(routing.websocket_urlpatterns)
        )
    ),
})
```

Settings update (settings.py):

python

```python
INSTALLED_APPS = [
    'daphne',  # Add at the top
    'django.contrib.admin',
    # ... other apps
    'channels',
    'apps.notifications',
]

ASGI_APPLICATION = 'config.asgi.application'

CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            "hosts": [('127.0.0.1', 6379)],
        },
    },
}
```

WebSocket Consumer (notifications/consumers.py):

python

```python
from channels.generic.websocket import AsyncWebsocketConsumer
import json

class NotificationConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.user = self.scope["user"]

        if self.user.is_authenticated:
            self.group_name = f"user_{self.user.id}"

            await self.channel_layer.group_add(
                self.group_name,
                self.channel_name
            )
            await self.accept()
        else:
            await self.close()

    async def disconnect(self, close_code):
        if hasattr(self, 'group_name'):
            await self.channel_layer.group_discard(
                self.group_name,
                self.channel_name
            )

    async def receive(self, text_data):
        """Handle incoming WebSocket messages"""
        data = json.loads(text_data)

        if data.get('action') == 'mark_read':
            notification_id = data.get('notification_id')
            # Mark notification as read (implement in view)

    async def notification_message(self, event):
        """Send notification to WebSocket"""
        await self.send(text_data=json.dumps({
            'type': 'notification',
            'notification': event['notification']
        }))
```

Routing (notifications/routing.py):

python

```python
from django.urls import path
from . import consumers
```

```
websocket_urlpatterns = [
    path('ws/notifications/', consumers.NotificationConsumer.as_asgi()),
]
```

Notification Service (notifications/services.py):

python

```python
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync
from .models import Notification
from django.contrib.auth import get_user_model

User = get_user_model()

class NotificationService:
    """Service class for handling notifications with OOP principles"""

    @staticmethod
    def send_notification(user, notification_type, title, message, related_object=None):
        """Create and send notification via WebSocket"""

        # Create notification in database
        notification = Notification.objects.create(
            recipient=user,
            notification_type=notification_type,
            title=title,
            message=message,
            related_object_type=related_object.__class__.__name__ if related_object else None,
            related_object_id=related_object.id if related_object else None
        )

        # Send via WebSocket
        channel_layer = get_channel_layer()
        async_to_sync(channel_layer.group_send)(
            f"user_{user.id}",
            {
                "type": "notification_message",
                "notification": {
                    "id": notification.id,
                    "type": notification_type,
                    "title": title,
                    "message": message,
                    "created_at": notification.created_at.isoformat(),
                    "is_read": False
                }
            }
```

```
        )

        return notification

    @staticmethod
    def notify_students_exam_scheduled(session):
        """Notify all students when exam is scheduled"""
        if not session.is_exam:
            return

        for group in session.groups.all():
            students = group.student_set.all()
            for student in students:
                NotificationService.send_notification(
                    user=student.user,
                    notification_type='exam_scheduled',
                    title=f"Examen programmé: {session.subject}",
                    message=f"Un examen de {session.subject} a été programmé le {session.start_datetime.strftime('%d/%m/%Y à %H:%M')} dans la salle {session.room.name}.",
                    related_object=session
                )

    @staticmethod
    def notify_students_reservation_approved(reservation_request):
        """Notify students when teacher reservation is approved"""
        if not reservation_request.teacher:
            return

        # Get all groups taught by this teacher
        sessions = Session.objects.filter(teacher=reservation_request.teacher)
        students = set()

        for session in sessions:
            for group in session.groups.all():
                students.update(group.student_set.all())

        for student in students:
            NotificationService.send_notification(
                user=student.user,
                notification_type='reservation_approved',
                title=f"Nouvelle séance: {reservation_request.teacher.user.get_full_name()}",
                message=f"Votre enseignant a réservé la salle {reservation_request.room.name} le {reservation_request.requested_datetime.strftime('%d/%m/%Y à %H:%M')} pour une durée de {reservation_request.duration}h. Motif: {reservation_request.reason}",
```

```
            related_object=reservation_request
        )

    @staticmethod
    def notify_admins_contact_received(contact_message):
        """Notify administrators of new contact message"""
        admins = User.objects.filter(role='admin')

        for admin in admins:
            NotificationService.send_notification(
                user=admin,
                notification_type='contact_received',
                title=f"Nouveau message: {contact_message.subject}",
                message=f"De: {contact_message.sender_name} ({contact_message.sender_email})",
                related_object=contact_message
            )
```

Base Template with WebSocket (base.html):

xml

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Gestion Emplois du Temps{% endblock %}</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    {% block extra_css %}{% endblock %}
</head>
<body>
    <!-- Navigation -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
        <div class="container-fluid">
            <a class="navbar-brand" href="{% url 'home' %}">
                <i class="fas fa-calendar-alt"></i> Emplois du Temps
            </a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    {% if user.is_authenticated %}
                        <li class="nav-item dropdown">
                            <a class="nav-link" href="#" id="notificationDropdown" data-bs-toggle="dropdown">
                                <i class="fas fa-bell"></i>
```

```
                <span class="badge bg-danger" id="notification-badge">0</span>
            </a>
            <ul class="dropdown-menu dropdown-menu-end" id="notification-list" style="width: 350px; max-height: 400px; overflow-y: auto;">
                <li><h6 class="dropdown-header">Notifications</h6></li>
                <li><hr class="dropdown-divider"></li>
                <li><p class="text-center text-muted">Chargement...</p></li>
            </ul>
        </li>
        <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" id="userDropdown" data-bs-toggle="dropdown">
                <i class="fas fa-user"></i> {{ user.get_full_name }}
            </a>
            <ul class="dropdown-menu">
                <li><a class="dropdown-item" href="{% url 'profile' %}">Mon Profil</a></li>
                <li><hr class="dropdown-divider"></li>
                <li><a class="dropdown-item" href="{% url 'logout' %}">Déconnexion</a></li>
            </ul>
        </li>
    {% else %}
        <li class="nav-item"><a class="nav-link" href="{% url 'login' %}">Connexion</a></li>
        <li class="nav-item"><a class="nav-link" href="{% url 'register' %}">Inscription</a></li>
    {% endif %}
    <li class="nav-item"><a class="nav-link" href="{% url 'contact' %}">Contact</a></li>
  </ul>
 </div>
</div>
</nav>

<!-- Messages -->
{% if messages %}
<div class="container mt-3">
    {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade show" role="alert">
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    {% endfor %}
</div>
{% endif %}

<!-- Toast Container for Real-time Notifications -->
<div class="position-fixed bottom-0 end-0 p-3" style="z-index: 11">
    <div id="notification-container"></div>
```

```
</div>

<!-- Main Content -->
{% block content %}{% endblock %}

<!-- Scripts -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

{% if user.is_authenticated %}
<script>
    // WebSocket connection for real-time notifications
    const userId = {{ user.id }};
    const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
    const notificationSocket = new WebSocket(
        protocol + '//' + window.location.host + '/ws/notifications/'
    );

    notificationSocket.onopen = function(e) {
        console.log('WebSocket connection established');
        loadNotifications();
    };

    notificationSocket.onmessage = function(e) {
        const data = JSON.parse(e.data);

        if (data.type === 'notification') {
            showToastNotification(data.notification);
            updateNotificationBell();
            addNotificationToList(data.notification);
        }
    };

    notificationSocket.onclose = function(e) {
        console.error('WebSocket closed unexpectedly');
        // Attempt reconnection after 5 seconds
        setTimeout(function() {
            location.reload();
        }, 5000);
    };

    notificationSocket.onerror = function(err) {
        console.error('WebSocket error:', err);
    };
```

```
function showToastNotification(notification) {
    const toast = `
        <div class="toast show" role="alert" data-bs-autohide="true" data-bs-delay="5000">
            <div class="toast-header bg-info text-white">
                <i class="fas fa-bell me-2"></i>
                <strong class="me-auto">${notification.title}</strong>
                <small>À l'instant</small>
                <button type="button" class="btn-close btn-close-white" data-bs-dismiss="toast"></button>
            </div>
            <div class="toast-body">
                ${notification.message}
            </div>
        </div>
    `;

    document.getElementById('notification-container').insertAdjacentHTML('beforeend', toast);

    // Auto-remove after delay
    setTimeout(() => {
        const toasts = document.querySelectorAll('.toast');
        if (toasts.length > 0) {
            toasts.remove();
        }
    }, 5000);
}

function updateNotificationBell() {
    fetch('/api/notifications/unread-count/')
        .then(response => response.json())
        .then(data => {
            const badge = document.getElementById('notification-badge');
            badge.textContent = data.count;
            badge.style.display = data.count > 0 ? 'inline' : 'none';
        });
}

function loadNotifications() {
    fetch('/api/notifications/recent/')
        .then(response => response.json())
        .then(data => {
            const list = document.getElementById('notification-list');
            list.innerHTML = '<li><h6 class="dropdown-header">Notifications</h6></li><li><hr class="dropdown-divider"></li>';
```

```
            if (data.notifications.length === 0) {
                list.innerHTML += '<li><p class="text-center text-muted p-3">Aucune notification</p></li>';
            } else {
                data.notifications.forEach(notif => {
                    addNotificationToList(notif);
                });
            }

            updateNotificationBell();
        });
    }

    function addNotificationToList(notification) {
        const list = document.getElementById('notification-list');
        const notifHTML = `
            <li>
                <a class="dropdown-item ${notification.is_read ? '' : 'fw-bold'}" href="#" onclick="markAsRead(${notification.id})">
                    <div class="d-flex align-items-start">
                        <i class="fas fa-circle text-primary me-2" style="font-size: 8px; margin-top: 6px;"></i>
                        <div>
                            <div class="fw-bold">${notification.title}</div>
                            <small class="text-muted">${notification.message}</small>
                            <div><small class="text-muted">${formatDate(notification.created_at)}</small></div>
                        </div>
                    </div>
                </a>
            </li>
            <li><hr class="dropdown-divider"></li>
        `;

        list.insertAdjacentHTML('beforeend', notifHTML);
    }

    function markAsRead(notificationId) {
        fetch(`/api/notifications/${notificationId}/mark-read/`, {
            method: 'POST',
            headers: {
                'X-CSRFToken': getCookie('csrftoken')
            }
        }).then(() => {
            updateNotificationBell();
        });
```

```
        }

        function formatDate(dateString) {
            const date = new Date(dateString);
            const now = new Date();
            const diff = Math.floor((now - date) / 1000);

            if (diff < 60) return "À l'instant";
            if (diff < 3600) return `Il y a ${Math.floor(diff / 60)} min`;
            if (diff < 86400) return `Il y a ${Math.floor(diff / 3600)} h`;
            return date.toLocaleDateString('fr-FR');
        }

        function getCookie(name) {
            let cookieValue = null;
            if (document.cookie && document.cookie !== '') {
                const cookies = document.cookie.split(';');
                for (let i = 0; i < cookies.length; i++) {
                    const cookie = cookies[i].trim();
                    if (cookie.substring(0, name.length + 1) === (name + '=')) {
                        cookieValue = decodeURIComponent(cookie.substring(name.length + 1));
                        break;
                    }
                }
            }
            return cookieValue;
        }
    </script>
    {% endif %}

    {% block extra_js %}{% endblock %}
</body>
</html>
```

Feature 3: ROOM RESERVATION REQUEST (Teacher/Association)

Form:

python

```
class RoomReservationForm(forms.ModelForm):
    class Meta:
        model = RoomReservationRequest
        fields = ['room', 'requested_datetime', 'duration', 'reason', 'is_exam']
        labels = {
            'room': 'Salle souhaitée',
            'requested_datetime': 'Date et heure',
            'duration': 'Durée (en heures)',
```

```
        'reason': 'Motif de la demande',
        'is_exam': 'Réservation pour un examen'
    }
    help_texts = {
        'reason': 'Veuillez préciser le motif de votre réservation',
        'is_exam': 'Cocher si cette réservation concerne un examen (les étudiants seront notifiés)'
    }
    widgets = {
        'requested_datetime': forms.DateTimeInput(attrs={
            'type': 'datetime-local',
            'class': 'form-control'
        }),
        'duration': forms.NumberInput(attrs={
            'min': 1,
            'max': 8,
            'class': 'form-control'
        }),
        'reason': forms.Textarea(attrs={
            'rows': 4,
            'class': 'form-control',
            'placeholder': 'Ex: Séance de rattrapage pour le groupe X'
        }),
        'room': forms.Select(attrs={'class': 'form-select'}),
        'is_exam': forms.CheckboxInput(attrs={'class': 'form-check-input'})
    }
```

View:

python

```
class RoomReservationCreateView(LoginRequiredMixin, CreateView):
    model = RoomReservationRequest
    form_class = RoomReservationForm
    template_name = 'scheduling/reservation_form.html'
    success_url = reverse_lazy('dashboard')

    def form_valid(self, form):
        user = self.request.user

        # Set requester based on role
        if user.role == 'teacher':
            form.instance.requester_type = 'teacher'
            form.instance.teacher = user.teacher
        elif user.role == 'association':
            form.instance.requester_type = 'association'
            form.instance.association = user.association
        else:
```

```
            messages.error(self.request, "Vous n'avez pas la permission de réserver une salle.")
            return redirect('dashboard')

        # Check for conflicts
        room = form.instance.room
        start_time = form.instance.requested_datetime
        end_time = start_time + timedelta(hours=form.instance.duration)

        if not room.check_availability(start_time, end_time):
            messages.error(self.request, 'La salle est déjà occupée à ce créneau.')
            return self.form_invalid(form)

        messages.success(self.request, 'Votre demande de réservation a été soumise avec succès.')
        return super().form_valid(form)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['available_rooms'] = Room.objects.filter(is_active=True).order_by('building', 'name')
        return context
```

DTL Template:

xml

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}

{% block content %}
<div class="container mt-5">
    <div class="row">
        <div class="col-md-8 offset-md-2">
            <div class="card">
                <div class="card-header bg-primary text-white">
                    <h3><i class="fas fa-calendar-plus"></i> Demande de Réservation de Salle</h3>
                </div>
                <div class="card-body">
                    <form method="post">
                        {% csrf_token %}

                        {{ form.room|as_crispy_field }}

                        <div class="row">
                            <div class="col-md-8">
                                {{ form.requested_datetime|as_crispy_field }}
                            </div>
                            <div class="col-md-4">
                                {{ form.duration|as_crispy_field }}
```

```
                </div>
            </div>

            {{ form.reason|as_crispy_field }}

            <div class="form-check mb-3">
                {{ form.is_exam }}
                <label class="form-check-label" for="{{ form.is_exam.id_for_label }}">
                    {{ form.is_exam.label }}
                </label>
                <div class="form-text">{{ form.is_exam.help_text }}</div>
            </div>

            <div class="alert alert-info">
                <i class="fas fa-info-circle"></i>
                Votre demande sera soumise à l'approbation de l'administrateur.
                Vous recevrez une notification une fois traitée.
            </div>

            <div class="d-grid gap-2">
                <button type="submit" class="btn btn-primary btn-lg">
                    <i class="fas fa-paper-plane"></i> Soumettre la Demande
                </button>
                <a href="{% url 'dashboard' %}" class="btn btn-secondary">
                    <i class="fas fa-times"></i> Annuler
                </a>
            </div>
        </form>
    </div>
  </div>
 </div>
</div>
</div>
{% endblock %}
```

Feature 4: CONTACT FORM (All Users including Guests)

Form:

python

```python
class ContactForm(forms.ModelForm):
    class Meta:
        model = ContactMessage
        fields = ['sender_name', 'sender_email', 'subject', 'message']
        labels = {
            'sender_name': 'Votre nom',
            'sender_email': 'Votre email',
```

```
        'subject': 'Sujet',
        'message': 'Message'
    }
    widgets = {
        'sender_name': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Nom complet'}),
        'sender_email': forms.EmailInput(attrs={'class': 'form-control', 'placeholder': 'email@exemple.com'}),
        'subject': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Sujet de votre message'}),
        'message': forms.Textarea(attrs={
            'class': 'form-control',
            'rows': 6,
            'placeholder': 'Décrivez votre demande ou question...'
        })
    }
```

View:

python

```
class ContactSubmitView(CreateView):
    model = ContactMessage
    form_class = ContactForm
    template_name = 'core/contact.html'
    success_url = reverse_lazy('contact_success')

    def form_valid(self, form):
        # Link to user if authenticated
        if self.request.user.is_authenticated:
            form.instance.user = self.request.user
            form.instance.sender_name = self.request.user.get_full_name()
            form.instance.sender_email = self.request.user.email

        response = super().form_valid(form)

        # Notify administrators
        from apps.notifications.services import NotificationService
        NotificationService.notify_admins_contact_received(self.object)

        messages.success(
            self.request,
            'Votre message a été envoyé avec succès. Nous vous répondrons dans les plus brefs délais.'
        )
        return response
```

DTL Template:

xml

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}
```

```
{% block content %}
<div class="container mt-5">
    <div class="row">
        <div class="col-md-8 offset-md-2">
            <div class="card">
                <div class="card-header bg-primary text-white">
                    <h3><i class="fas fa-envelope"></i> Contactez l'Administration</h3>
                </div>
                <div class="card-body">
                    <p class="text-muted">
                        Vous avez une question, une suggestion ou besoin d'aide?
                        Envoyez-nous un message et nous vous répondrons rapidement.
                    </p>

                    <form method="post">
                        {% csrf_token %}

                        {% if not user.is_authenticated %}
                            {{ form.sender_name|as_crispy_field }}
                            {{ form.sender_email|as_crispy_field }}
                        {% endif %}

                        {{ form.subject|as_crispy_field }}
                        {{ form.message|as_crispy_figure }}

                        <div class="d-grid">
                            <button type="submit" class="btn btn-primary btn-lg">
                                <i class="fas fa-paper-plane"></i> Envoyer le Message
                            </button>
                        </div>
                    </form>
                </div>
                <div class="card-footer text-muted">
                    <i class="fas fa-clock"></i> Temps de réponse moyen: 24-48 heures
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

Feature 5: PUBLIC LANDING PAGE (Guests)

View:

python

```
class LandingPageView(TemplateView):
```

```
    template_name = 'public/landing.html'

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['departments'] = Program.objects.values_list('department', flat=True).distinct()
        context['total_rooms'] = Room.objects.filter(is_active=True).count()
        context['total_programs'] = Program.objects.count()
        return context
```

DTL Template:

xml

```
{% extends 'base.html' %}

{% block content %}
<!-- Hero Section -->
<div class="bg-primary text-white py-5">
    <div class="container">
        <div class="row align-items-center">
            <div class="col-md-8">
                <h1 class="display-4"><i class="fas fa-graduation-cap"></i> Gestion Intelligente des Emplois du Temps</h1>
                <p class="lead">Système automatisé de planification universitaire</p>
                <div class="mt-4">
                    {% if not user.is_authenticated %}
                        <a href="{% url 'register' %}" class="btn btn-light btn-lg me-2">
                            <i class="fas fa-user-plus"></i> S'inscrire
                        </a>
                        <a href="{% url 'login' %}" class="btn btn-outline-light btn-lg">
                            <i class="fas fa-sign-in-alt"></i> Se Connecter
                        </a>
                    {% else %}
                        <a href="{% url 'dashboard' %}" class="btn btn-light btn-lg">
                            <i class="fas fa-tachometer-alt"></i> Tableau de Bord
                        </a>
                    {% endif %}
                </div>
            </div>
            <div class="col-md-4 text-center">
                <i class="fas fa-calendar-alt" style="font-size: 150px; opacity: 0.3;"></i>
            </div>
        </div>
    </div>
</div>

<!-- Features Section -->
<div class="container my-5">
```

```
    <h2 class="text-center mb-5">Fonctionnalités Principales</h2>
    <div class="row">
        <div class="col-md-4 mb-4">
            <div class="card h-100">
                <div class="card-body text-center">
                    <i class="fas fa-robot fa-3x text-primary mb-3"></i>
                    <h5>Génération Automatique</h5>
                    <p>Création intelligente d'emplois du temps optimisés selon les contraintes</p>
                </div>
            </div>
        </div>
        <div class="col-md-4 mb-4">
            <div class="card h-100">
                <div class="card-body text-center">
                    <i class="fas fa-bell fa-3x text-primary mb-3"></i>
                    <h5>Notifications en Temps Réel</h5>
                    <p>Recevez instantanément les mises à jour importantes</p>
                </div>
            </div>
        </div>
        <div class="col-md-4 mb-4">
            <div class="card h-100">
                <div class="card-body text-center">
                    <i class="fas fa-door-open fa-3x text-primary mb-3"></i>
                    <h5>Réservation de Salles</h5>
                    <p>Système de demande et validation de réservations</p>
                </div>
            </div>
        </div>
    </div>
</div>

<!-- Stats Section -->
<div class="bg-light py-5">
    <div class="container">
        <div class="row text-center">
            <div class="col-md-4">
                <h2 class="text-primary">{{ total_rooms }}</h2>
                <p>Salles Disponibles</p>
            </div>
            <div class="col-md-4">
                <h2 class="text-primary">{{ total_programs }}</h2>
                <p>Filières</p>
            </div>
```

```
            <div class="col-md-4">
                <h2 class="text-primary">{{ departments|length }}</h2>
                <p>Départements</p>
            </div>
        </div>
    </div>
</div>

<!-- CTA Section -->
<div class="container my-5 text-center">
    <h2>Prêt à Commencer?</h2>
    <p class="lead">Rejoignez notre plateforme de gestion d'emplois du temps</p>
    <div class="mt-4">
        <a href="{% url 'public_room_search' %}" class="btn btn-primary btn-lg me-2">
            <i class="fas fa-search"></i> Rechercher une Salle
        </a>
        <a href="{% url 'contact' %}" class="btn btn-outline-primary btn-lg">
            <i class="fas fa-envelope"></i> Nous Contacter
        </a>
    </div>
</div>
{% endblock %}
```

Feature 6: REGISTRATION FORMS

Student Registration:

python

```
class StudentRegistrationForm(UserCreationForm):
    first_name = forms.CharField(max_length=100, required=True, label="Prénom")
    last_name = forms.CharField(max_length=100, required=True, label="Nom")
    email = forms.EmailField(required=True, label="Email universitaire")
    student_id = forms.CharField(max_length=20, required=True, label="Numéro d'étudiant")
    program = forms.ModelChoiceField(
        queryset=Program.objects.all(),
        required=True,
        label="Filière",
        widget=forms.Select(attrs={'class': 'form-select'})
    )
    group = forms.ModelChoiceField(
        queryset=Group.objects.none(),
        required=True,
        label="Groupe",
        widget=forms.Select(attrs={'class': 'form-select'})
    )

    class Meta:
```

```
        model = User
        fields = ['username', 'first_name', 'last_name', 'email', 'password1', 'password2']

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if 'program' in self.data:
            try:
                program_id = int(self.data.get('program'))
                self.fields['group'].queryset = Group.objects.filter(program_id=program_id)
            except (ValueError, TypeError):
                pass

    def save(self, commit=True):
        user = super().save(commit=False)
        user.role = 'student'
        if commit:
            user.save()
            Student.objects.create(
                user=user,
                student_id=self.cleaned_data['student_id'],
                enrollment_year=timezone.now().year,
                group=self.cleaned_data['group']
            )
        return user
```

Association Registration:

python

```
class AssociationRegistrationForm(UserCreationForm):
    association_name = forms.CharField(
        max_length=200,
        required=True,
        label="Nom de l'association"
    )
    description = forms.CharField(
        widget=forms.Textarea(attrs={'rows': 4}),
        required=True,
        label="Description de l'association"
    )
    president_name = forms.CharField(
        max_length=100,
        required=True,
        label="Nom du président"
    )
    email = forms.EmailField(required=True, label="Email de l'association")
    phone = forms.CharField(max_length=20, required=True, label="Téléphone")
```

```python
    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

    def save(self, commit=True):
        user = super().save(commit=False)
        user.role = 'association'
        user.is_active = False  # Requires admin approval
        if commit:
            user.save()
            Association.objects.create(
                user=user,
                name=self.cleaned_data['association_name'],
                description=self.cleaned_data['description'],
                president_name=self.cleaned_data['president_name'],
                email=self.cleaned_data['email'],
                phone=self.cleaned_data['phone'],
                is_approved=False
            )
        return user
```

FAKE DATA GENERATION

Management Command (management/commands/populate_db.py):

python

```python
from django.core.management.base import BaseCommand
from django.contrib.auth import get_user_model
from apps.core.models import *
from apps.scheduling.models import *
from faker import Faker
import random
from datetime import datetime, timedelta

User = get_user_model()
fake = Faker('fr_FR')

class Command(BaseCommand):
    help = 'Populate database with fake data for testing'

    def handle(self, *args, **options):
        self.stdout.write('Starting data population...')

        # Create admin user
        admin, created = User.objects.get_or_create(
            username='admin',
```

```
    defaults={
        'email': 'admin@university.ma',
        'role': 'admin',
        'first_name': 'Admin',
        'last_name': 'System',
        'is_staff': True,
        'is_superuser': True
    }
)
if created:
    admin.set_password('admin123')
    admin.save()
    self.stdout.write(self.style.SUCCESS(f'✓ Admin created'))

# Create equipment
equipment_list = [
    'Projecteur', 'Tableau blanc', 'Ordinateurs (30 postes)',
    'Climatisation', 'Tableau interactif', 'Connexion Internet',
    'Microphone', 'Système audio', 'Matériel de laboratoire'
]
equipments = []
for eq_name in equipment_list:
    eq, _ = Equipment.objects.get_or_create(
        name=eq_name,
        defaults={'description': f'Équipement: {eq_name}'}
    )
    equipments.append(eq)
self.stdout.write(self.style.SUCCESS(f'✓ {len(equipments)} equipments created'))

# Create rooms in bulk (example: B1-B23, C1-C15, etc.)
buildings = [
    {'prefix': 'B', 'start': 1, 'end': 23, 'type': 'classe', 'capacity': 40},
    {'prefix': 'C', 'start': 1, 'end': 15, 'type': 'classe', 'capacity': 30},
    {'prefix': 'Amphi', 'start': 1, 'end': 5, 'type': 'amphitheatre', 'capacity': 200},
    {'prefix': 'Labo', 'start': 1, 'end': 10, 'type': 'labo', 'capacity': 25},
]

rooms = []
for building in buildings:
    for num in range(building['start'], building['end'] + 1):
        room = Room.objects.create(
            name=f'{building['prefix']}{num}',
            room_type=building['type'],
```

```
            capacity=building['capacity'],
            building=f"Bâtiment {building['prefix']}",
            floor=random.randint(0, 3),
            is_active=True
        )
        # Add random equipment
        room.equipment.set(random.sample(equipments, k=random.randint(2, 5)))
        rooms.append(room)

self.stdout.write(self.style.SUCCESS(f'✓ {len(rooms)} rooms created'))

# Create programs
programs_data = [
    {'name': 'Licence Analytique de Données', 'code': 'LAD', 'dept': 'Informatique', 'level': 'licence'},
    {'name': 'Master Data Science', 'code': 'MDS', 'dept': 'Informatique', 'level': 'master'},
    {'name': 'Licence Mathématiques', 'code': 'LM', 'dept': 'Mathématiques', 'level': 'licence'},
    {'name': 'Licence Physique', 'code': 'LP', 'dept': 'Physique', 'level': 'licence'},
    {'name': 'Master Intelligence Artificielle', 'code': 'MIA', 'dept': 'Informatique', 'level': 'master'},
]

programs = []
for prog_data in programs_data:
    prog, _ = Program.objects.get_or_create(
        code=prog_data['code'],
        defaults={
            'name': prog_data['name'],
            'department': prog_data['dept'],
            'degree_level': prog_data['level']
        }
    )
    programs.append(prog)

self.stdout.write(self.style.SUCCESS(f'✓ {len(programs)} programs created'))

# Create groups
groups = []
for program in programs:
    for group_num in range(1, 4):  # 3 groups per program
        group = Group.objects.create(
            name=f"Groupe {group_num}",
            program=program,
            academic_year='2025/2026',
            capacity=random.randint(30, 50)
```

```
        )
        groups.append(group)

self.stdout.write(self.style.SUCCESS(f'✓ {len(groups)} groups created'))

# Create teachers
departments = ['Informatique', 'Mathématiques', 'Physique', 'Chimie']
teachers = []
for i in range(30):
    user = User.objects.create_user(
        username=f'teacher{i+1}',
        email=f'teacher{i+1}@university.ma',
        password='teacher123',
        first_name=fake.first_name(),
        last_name=fake.last_name(),
        role='teacher'
    )
    teacher = Teacher.objects.create(
        user=user,
        specialization=fake.job(),
        department=random.choice(departments),
        employee_id=f'T{1000+i}'
    )
    teachers.append(teacher)

self.stdout.write(self.style.SUCCESS(f'✓ {len(teachers)} teachers created'))

# Create students
for i in range(100):
    user = User.objects.create_user(
        username=f'student{i+1}',
        email=f'student{i+1}@university.ma',
        password='student123',
        first_name=fake.first_name(),
        last_name=fake.last_name(),
        role='student'
    )
    Student.objects.create(
        user=user,
        student_id=f'S{20000+i}',
        enrollment_year=random.choice(),
        group=random.choice(groups)
    )
```

```
self.stdout.write(self.style.SUCCESS(f'✓ 100 students created'))

# Create associations
association_names = [
    'Club Informatique', 'Association Culturelle', 'Club Sportif',
    'Association Scientifique', 'Club Artistique'
]
for i, assoc_name in enumerate(association_names):
    user = User.objects.create_user(
        username=f'association{i+1}',
        email=f'association{i+1}@university.ma',
        password='assoc123',
        role='association'
    )
    Association.objects.create(
        user=user,
        name=assoc_name,
        description=fake.text(max_nb_chars=200),
        president_name=fake.name(),
        email=f'{assoc_name.lower().replace(" ", "")}@university.ma',
        phone=fake.phone_number(),
        is_approved=True
    )

self.stdout.write(self.style.SUCCESS(f'✓ {len(association_names)} associations created'))

# Create sessions
subjects = [
    'Algorithmique', 'Base de Données', 'Programmation Python',
    'Intelligence Artificielle', 'Analyse Mathématique',
    'Algèbre Linéaire', 'Physique Quantique', 'Statistiques'
]

session_types = ['cours', 'td', 'tp']
start_date = datetime.now().replace(hour=8, minute=0, second=0, microsecond=0)

for _ in range(200):
    session_type = random.choice(session_types)
    duration = 2 if session_type == 'cours' else 1.5

    session = Session.objects.create(
        session_type=session_type,
```

```
            subject=random.choice(subjects),
            teacher=random.choice(teachers),
            room=random.choice(rooms),
            start_datetime=start_date + timedelta(days=random.randint(0, 30), hours=random.randint(0, 8)),
            end_datetime=start_date + timedelta(days=random.randint(0, 30), hours=random.randint(0, 8)) + timedelta(hours=duration),
            is_exam=random.random() < 0.1,  # 10% chance of exam
            is_validated=True
        )
        session.groups.set(random.sample(groups, k=random.randint(1, 2)))

    self.stdout.write(self.style.SUCCESS(f'✓ 200 sessions created'))

    # Create some pending reservation requests
    for _ in range(15):
        requester_type = random.choice(['teacher', 'association'])
        req = RoomReservationRequest.objects.create(
            requester_type=requester_type,
            teacher=random.choice(teachers) if requester_type == 'teacher' else None,
            association=Association.objects.order_by('?').first() if requester_type == 'association' else None,
            room=random.choice(rooms),
            requested_datetime=datetime.now() + timedelta(days=random.randint(1, 30)),
            duration=random.randint(1, 4),
            reason=fake.sentence(),
            status=random.choice(['pending', 'approved', 'rejected']),
            is_exam=random.random() < 0.2
        )

    self.stdout.write(self.style.SUCCESS(f'✓ 15 reservation requests created'))

    self.stdout.write(self.style.SUCCESS('✅ Database population completed successfully!'))
    self.stdout.write(self.style.SUCCESS('Login credentials:'))
    self.stdout.write(self.style.SUCCESS('  Admin: admin / admin123'))
    self.stdout.write(self.style.SUCCESS('  Teacher: teacher1 / teacher123'))
    self.stdout.write(self.style.SUCCESS('  Student: student1 / student123'))
    self.stdout.write(self.style.SUCCESS('  Association: association1 / assoc123'))
```

URLS CONFIGURATION

Main urls.py:

python

```
from django.contrib import admin
from django.urls import path, include
from apps.public.views import LandingPageView

urlpatterns = [
```

```
    path('admin/', admin.site.urls),
    path('', LandingPageView.as_view(), name='home'),
    path('accounts/', include('apps.accounts.urls')),
    path('core/', include('apps.core.urls')),
    path('scheduling/', include('apps.scheduling.urls')),
    path('api/notifications/', include('apps.notifications.urls')),
]
```

REQUIREMENTS.TXT

text

```
Django>=5.0
psycopg2-binary>=2.9
Pillow>=10.0
reportlab>=4.0
openpyxl>=3.1
factory-boy>=3.3
Faker>=20.0
django-widget-tweaks>=1.5
python-decouple>=3.8
django-crispy-forms>=2.1
crispy-bootstrap5>=2.0
channels>=4.0
channels-redis>=4.1
daphne>=4.0
redis>=5.0
celery>=5.3
django-celery-beat>=2.5
```

PROJECT STRUCTURE

text

```
gestion_emplois_temps/
├── manage.py
├── requirements.txt
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── apps/
│   ├── __init__.py
│   ├── accounts/
│   │   ├── models.py (User, Teacher, Student, Association)
│   │   ├── views.py
│   │   ├── forms.py
│   │   ├── urls.py
```

```
│   │   └── templates/accounts/
│   ├── core/
│   │   ├── models.py (Program, Group, Room, Equipment, ContactMessage)
│   │   ├── views.py
│   │   ├── forms.py
│   │   ├── urls.py
│   │   └── templates/core/
│   ├── scheduling/
│   │   ├── models.py (Session, Timetable, RoomReservationRequest, TeacherUnavailability)
│   │   ├── views.py
│   │   ├── forms.py
│   │   ├── services.py
│   │   ├── algorithms.py
│   │   ├── urls.py
│   │   └── templates/scheduling/
│   ├── notifications/
│   │   ├── models.py (Notification)
│   │   ├── consumers.py
│   │   ├── routing.py
│   │   ├── services.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   └── templates/notifications/
│   ├── public/
│   │   ├── views.py
│   │   └── templates/public/
│   └── utils/
│       ├── exporters.py
│       └── validators.py
├── static/
│   ├── css/
│   ├── js/
│   └── images/
└── templates/
    ├── base.html
    └── includes/
```

IMPLEMENTATION INSTRUCTIONS

1. Create Django project: django-admin startproject config .
2. Create all apps: python manage.py startapp accounts, etc.
3. Configure settings.py with French language and timezone
4. Implement all models with OOP principles
5. Create migrations: python manage.py makemigrations && python manage.py migrate
6. Create superuser: python manage.py createsuperuser
7. Run populate command: python manage.py populate_db

8. Install Redis: redis-server (for real-time notifications)
9. Run development server: daphne -b 0.0.0.0 -p 8000 config.asgi:application

KEY FEATURES CHECKLIST

- ✅ 5 User Roles (Admin, Teacher, Student, Association, Guest)
- ✅ Bulk Room Creation (B1-B23 in one operation)
- ✅ Real-time WebSocket Notifications
- ✅ Public Landing Page (no auth required)
- ✅ Student & Association Registration Forms
- ✅ Contact Form with Admin Notifications
- ✅ Room Reservation System (Teacher/Association)
- ✅ Exam Scheduling with Student Notifications
- ✅ Approval Workflow for Reservations
- ✅ All Content in French (UI labels, messages, forms)
- ✅ Django Template Language (DTL) Forms
- ✅ Bootstrap 5 Styling
- ✅ OOP Design Patterns
- ✅ Fake Data Generation (50+ rooms, 30 teachers, 100 students)
- ✅ Database with proper relationships

Generate this complete Django application following these specifications exactly.